AIS Graph Analytics

Архитектура и эксплуатация

Konstantin Lofichenko
2026-10-14

Текущая live, historical и графовая платформа. Автоматический запуск PyFlink, durable checkpoints и восстановление, derived streams в ClickHouse, шесть tabs Metabase.

Содержание

1. Обзор платформы 3

2. Архитектура 4

3. Docker runtime 8

4. Установка с чистого состояния 8

5. Модель данных ClickHouse 9

6. dbt 9

7. Сохранённый legacy historical ClickHouse путь 10

8. Ежедневная оркестрация Airflow 10

9. AI enrichment 11

10. Neo4j и графовая аналитика 11

11. Metabase 11

12. CI 12

13. Release workflow 12

14. Опубликованные containers 12

15. Проверка и эксплуатация 12

16. Важные правила качества данных 13

17. Известное ограничение: фрагментация port visit на границе полуночи 13

18. Troubleshooting 13

19. Безопасная остановка и cleanup 13

20. Карта документации репозитория 14

21. Текущее состояние и roadmap 14

Перегенерация PDF 14

Актуальная архитектура и эксплуатация платформы. Документация проверена 2026-10-14. Английское руководство, English PDF и Русский PDF описывают текущую реализацию. Инструкции конкретных компонентов находятся в их README; конфигурация в репозитории остаётся первичным источником параметров.

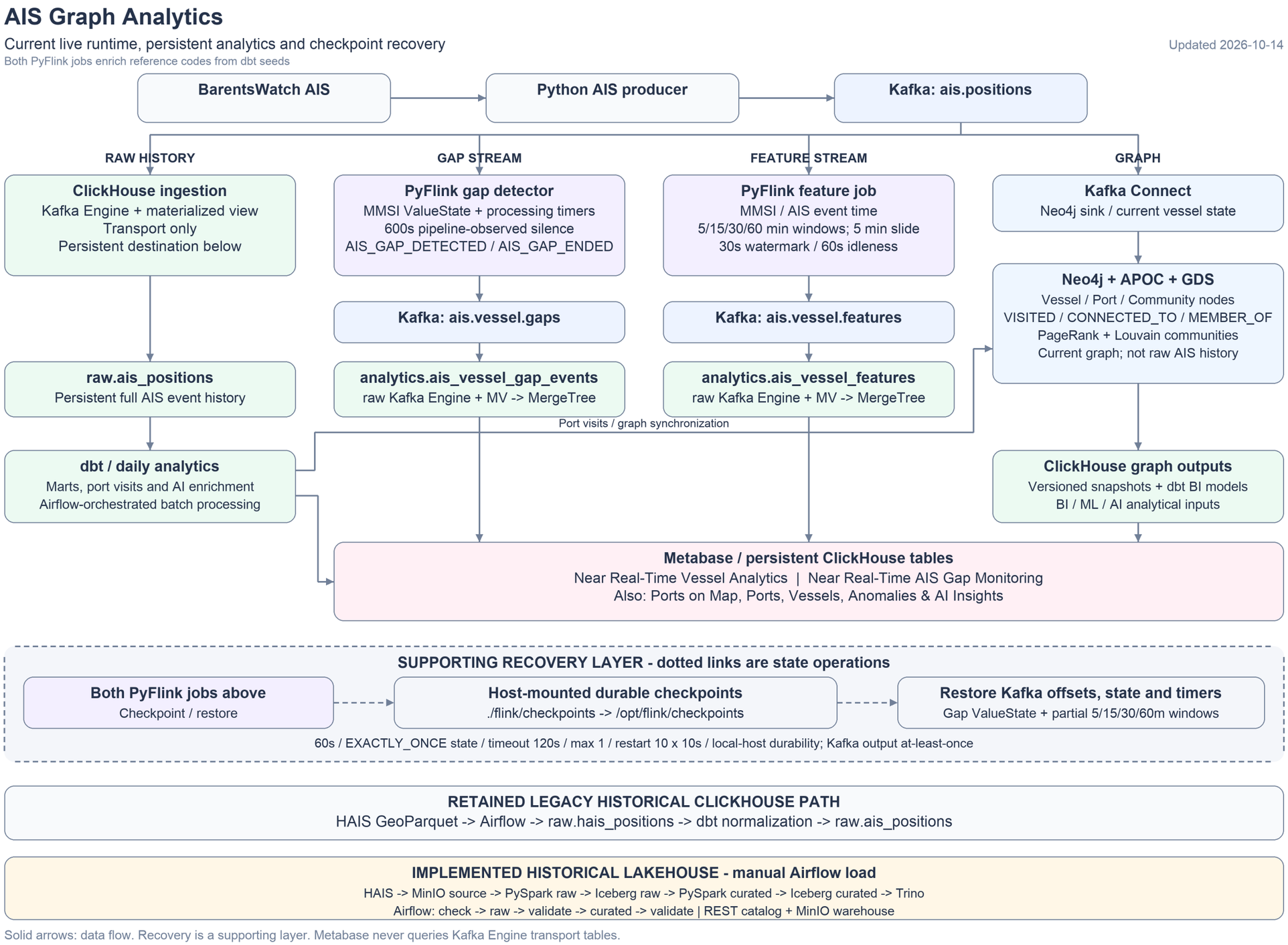
1. Обзор платформы

Платформа объединяет live AIS от BarentsWatch и исторические HAIS GeoParquet. Python producer передаёт live-сообщения в Kafka. ClickHouse хранит каноническую историю и аналитические результаты, dbt выполняет преобразования, Neo4j хранит текущее состояние графа, GDS рассчитывает графовые метрики, Airflow координирует суточную обработку, OpenAI объясняет выбранные аномалии, Metabase визуализирует данные. PyFlink 2.2.1 непрерывно рассчитывает окна признаков и события AIS-пауз.

Задача	Основная система
История AIS и аналитические таблицы	ClickHouse
Исторический lakehouse	MinIO, Iceberg REST, PySpark и Trino
SQL-преобразования	dbt на ClickHouse
Live transport	Kafka
Stateful stream processing	PyFlink 2.2.1
Vessel/Port/Community graph	Neo4j и GDS
Оркестрация batch и суточных задач	Airflow
Объяснение аномалий	OpenAI enrichment pipeline
BI	Metabase
Проверки и публикация образов	GitHub Actions и GHCR

2. Архитектура

Архитектура: live и исторический lakehouse



Масштабируемая SVG-диаграмма и исходник диаграммы. Сплошные стрелки показывают данные, пунктирный слой - checkpoint/restore. Реализованный исторический lakehouse в янтарной области работает локально через manual Airflow DAG.

2.1 Три live-пути

```
BarentsWatch -> Python AIS producer -> Kafka ais.positions
  -> ClickHouse Kafka Engine / MV -> raw.ais_positions
  -> PyFlink gap detector -> ais.vessel.gaps
     -> raw Kafka Engine / MV -> analytics.ais_vessel_gap_events
  -> PyFlink multi-window features -> ais.vessel.features
     -> raw Kafka Engine / MV -> analytics.ais_vessel_features
  -> Kafka Connect Neo4j sink -> текущие Vessel nodes
```

Producer использует MMSI как Kafka key и сохраняет поля AIS. ClickHouse отвечает за историю, Flink - за непрерывные вычисления с состоянием, Neo4j - за графовые связи и текущие узлы. Neo4j дополняет хранилище истории.

2.2 Flink: автоматический запуск и семантика

```
docker compose --profile streaming up -d
docker compose logs -f flink-job-submitter
curl -fsS http://localhost:8082/jobs/overview
```

Одноразовый flink-job-submitter ждёт доступный JobManager, два зарегистрированных TaskManager, Kafka protocol handshake и свободный slot для каждой отсутствующей задачи. При parallelism=1 каждой задаче нужен один slot. Readiness polling выполняется каждые две секунды до 300 секунд. Успешное завершение с кодом 0 происходит только после подтверждения RUNNING для обеих задач.

Job IDs сохраняются до submission в общем registry volume под файловой блокировкой; Flink получает фиксированный `$internal.pipeline.job-id`. Повторный запуск учитывает переходные и restarting состояния. Ранее вручную запущенные задачи принимаются по имени и source/operator plan, а не только имени. Несколько совпадающих активных задач вызывают ошибку. Submitter отправляет только отсутствующие задачи и использует `--pyFiles /opt/flink/jobs` для общего модуля reference data. Это startup-клиент, а не постоянный supervisor.

```
FLINK_GAP_TIMEOUT_SECONDS=600
FLINK_FEATURE_WINDOWS_MINUTES=5,15,30,60
FLINK_FEATURE_SLIDE_MINUTES=5
FLINK_FEATURE_WATERMARK_SECONDS=30
FLINK_FEATURE_IDLE_SECONDS=60
```

Gap detector хранит ValueState и processing-time timers по MMSI. После 600 секунд наблюдаемого отсутствия сообщений выдаётся один AIS_GAP_DETECTED. Следующее сообщение завершает активную паузу и выдаёт AIS_GAP_ENDED. Продолжительность включает весь период молчания, в том числе порог 600 секунд. msgtime описывает исходное AIS-событие; обнаружение определяется processing-time clock, а не watermark или возрастом исторического AIS timestamp.

Feature job использует один KafkaSource, event time из msgtime, bounded watermark 30 секунд и idleness 60 секунд. Четыре ветки окон 5/15/30/60 минут с шагом 5 минут объединяются в один output topic; window_minutes отличает ветки. Окна закрываются по watermark; дополнительная allowed lateness не задана. Поздние события за watermark не меняют уже закрытое окно. Имя судна - последнее non-null имя по времени события внутри окна. Ship type и navigation status берутся из последнего event-time observation, включая null. Speed count/mean/min/max сохраняют текущие правила допустимости значений. Reference labels читаются из dbt seeds

`ais_ship_types.csv` и `ais_navigational_status.csv`.

2.3 Durable checkpoints и восстановление

```
FLINK_CHECKPOINT_INTERVAL_SECONDS=60
FLINK_CHECKPOINT_TIMEOUT_SECONDS=120
FLINK_CHECKPOINT_MIN_PAUSE_SECONDS=10
FLINK_CHECKPOINT_DIR=file:///opt/flink/checkpoints
FLINK_RESTART_ATTEMPTS=10
FLINK_RESTART_DELAY_SECONDS=10
```

Обе задачи используют EXACTLY_ONCE checkpoint state mode, один одновременный checkpoint и retention on cancellation. Gitignored host-каталог

`./flink/checkpoints` доступен JobManager, обоим TaskManager и submitter по пути

`/opt/flink/checkpoints`; flink-checkpoint-init устанавливает права доступа. Подкаталоги gap/ и features/ изолируют состояния задач. Stable operator UIDs позволяют восстановить Kafka offsets, keyed ValueState, processing-time timers и незавершённые feature windows.

При отказе задач в живом кластере Flink использует fixed-delay restart: 10 попыток с интервалом 10 секунд и последний завершённый checkpoint. После полной потери JobManager submitter выбирает последний finalized `_metadata` и передаёт `-s` и `-claimMode NO_CLAIM`. Исходный snapshot сохраняется; новая задача создаёт независимый checkpoint. Ошибка выбранного restore не вызывает скрытый переход к пустому состоянию. При отсутствии checkpoint выводится предупреждение и выполняется fresh start с latest Kafka offsets.

Без восстановления checkpoint keyed state и timers теряются после полного перезапуска кластера; также теряются незавершённые окна. Kafka group offsets не заменяют checkpoint. Kafka должна сохранять offsets, требуемые snapshot. Локальный общий mount не обеспечивает multi-host HA или защиту от потери диска/ хоста. Нельзя удалять checkpoint files, пока они нужны текущей задаче или restore. Очистка старых snapshot требует отдельной проверки recovery dependencies.

EXACTLY_ONCE относится к состоянию Flink. Kafka sinks используют AT_LEAST_ONCE, а ClickHouse ingestion также допускает повторные записи. Replay после restore может повторить output records, включая gap events; end-to-end exactly-once и автоматическая дедупликация здесь не обеспечиваются.

```
# Перезапустить только Flink; остальные сервисы продолжают работать.
docker compose --profile streaming restart \
  flink-jobmanager flink-taskmanager flink-feature-taskmanager
# Для явного повторного reconciliation:
docker compose --profile streaming run --rm --no-deps flink-job-submitter
# Recreate только Flink, например после изменения конфигурации:
docker compose --profile streaming up -d --no-deps --force-recreate \
  flink-jobmanager flink-taskmanager flink-feature-taskmanager flink-job-submitter
```

Explicit Compose restart вызывает submitter через depends_on restart:true. После abrupt Docker failure необходимо снова выполнить Compose startup или reconciliation. При исчерпании Flink retries сначала устраните причину отказа. Перед restart проверьте completed checkpoints обоих jobs в UI http://localhost:8082 или `/jobs/<job-id>/checkpoints`.

```
# Только при отсутствии другой активной gap job; URI замените реальным snapshot.
docker compose exec -T flink-jobmanager /opt/flink/bin/flink run -d \
  -s file:///opt/flink/checkpoints/gap/<job-id>/chk-<id>/_metadata \
  -claimMode NO_CLAIM --pyFiles /opt/flink/jobs \
  -py /opt/flink/jobs/ais_gap_detector.py
```

Для feature job используйте features/ snapshot и ais_vessel_features.py. Обычные manual commands без `-s` начинают fresh state; для стандартного запуска предпочтителен submitter. Подробные команды и проверки: Flink README. Проверка 2026-10-14 подтвердила Flink-only restart, восстановление состояния/таймеров/окон, новые completed checkpoints и поступление результатов; Kafka, ClickHouse, Neo4j, Airflow и Metabase не перезапускались.

2.4 Исторический lakehouse и сохранённый legacy-путь

```
Legacy: HAIS -> Airflow -> raw.hais_positions -> dbt -> raw.ais_positions
Lakehouse: HAIS -> отдельная загрузка в MinIO -> PySpark -> Iceberg raw
  -> PySpark curated -> Iceberg curated -> Trino
Airflow: check_source_files -> ingest_raw_iceberg -> validate_raw
  -> curate_hais -> validate_curated -> final_summary
```

Lakehouse реализован локально. Старый ClickHouse/Airflow-путь сохранён без изменений. Live Kafka/Flink/ClickHouse/Neo4j/Metabase работает независимо. MinIO хранит source Parquet в `s3://ais-data/hais/source/`, а таблицы Iceberg - в отдельном `s3://warehouse/`. REST catalog использует persistent SQLite. PySpark выполняет raw ingestion и curated normalization; Trino даёт SQL-доступ. Airflow только запускает готовые jobs и validators, без загрузки host-файлов.

Raw: `iceberg.hais_raw.ais_positions`. Curated:

`iceberg.hais_curated.ais_positions`. Сохраняются все строки и binary geometry. Curated schema:

Колонка	Тип	NOT NULL
date_time	TIMESTAMP	Да
mmsi	INT	Да
longitude	DECIMAL(9,6)	Да
latitude	DECIMAL(8,6)	Да
navigational_status	INT	Да
course_over_ground	DECIMAL(5,2)	Нет
true_heading	INT	Нет
speed_over_ground	DECIMAL(5,2)	Нет
rate_of_turn	INT	Нет
maneuver	INT	Да
data_source	STRING	Да
ais_class	STRING	Да
message_type	INT	Да
geometry	BINARY	Да

Переименования: date_time_utc -> date_time, status -> navigational_status, maneuvre -> maneuver, msg_type -> message_type. Course сохраняется только при

`0 <= value < 360`; heading 511, speed 102.30 и turn -128 становятся NULL. Координаты округляются до шести десятичных знаков. Других бизнес-преобразований, фильтрации или дедупликации нет. Единственная partition - `day(date_time)`. TIMESTAMP в Trino отображается как timestamp(6) with time zone; geometry - varbinary.

```
docker compose --profile batch --profile lakehouse build airflow spark
docker compose --profile batch --profile lakehouse up -d --wait --wait-timeout 300 airflow minio minio-init
   iceberg-rest trino
# Сначала проверить HAIS_UPLOAD_HOST_DIR и состав dataset:
docker compose --profile lakehouse run --rm hais-upload
docker compose exec -T airflow airflow dags unpause ais_hais_lakehouse_load
docker compose exec -T airflow airflow dags trigger --run-id hais_manual_1 ais_hais_lakehouse_load
python3 lakehouse/validate_curated.py
```

DAG `ais_hais_lakehouse_load` запускается вручную: schedule=None, catchup=False, max_active_runs=1. Spark работает в отдельных контейнерах через DockerOperator, не внутри Airflow. Source check повторяется до двух раз, Spark loads - один раз, задержка две минуты; validators без retries. Ошибка блокирует downstream. Graph view и task logs доступны в Airflow UI.

Два полных manual runs проверены: 30 файлов за сентябрь 2026 и 23,295,797 строк в raw и curated. Повторный запуск не добавляет дубликаты. NULL counts: course 2,517,680; heading 4,511,883; speed 50,962; turn 4,221,794. Обязательные поля без NULL, sentinels отсутствуют, 30 дневных partitions подтверждены.

Trino: порт 8090, heap 8 GiB, query memory 4 GiB, total 6 GiB, headroom 2 GiB. REST: 8181; MinIO API: 9002; console: 9003. Старые порты не изменены. Ограничения: локальный диск, SQLite REST fixture, Docker socket даёт доступ к локальному daemon; нет distributed HA, incremental load или общей транзакции raw/curated. Старые snapshots могут занимать диск. Validation counts привязаны к сентябрю; нельзя запускать перекрывающиеся CLI/DAG

overwrite loads. MinIO собирается из pinned upstream source; Spark image нужно пересобирать после изменения jobs. Полные команды и параметры: lakehouse README.

2.5 Аналитический граф

```
raw.ais_positions -> dbt daily features -> anomalies -> AI -> enriched mart
raw.ais_positions -> port visits -> Neo4j -> PageRank / Louvain
  -> ClickHouse graph snapshots -> dbt views -> Metabase
```

Port visits создают Vessel-to-Port VISITED, последовательные посещения - Port-to-Port CONNECTED_TO, communities - Port-to-Community MEMBER_OF. PageRank/Louvain результаты выгружаются в ClickHouse как reusable BI/ML/AI inputs. Metabase читает сохранённые аналитические результаты.

3. Docker runtime

Profile	Сервисы и назначение
Без profile	Kafka, Kafka Connect, ksqlDB, Kafbat UI, ClickHouse, Neo4j
live	AIS producer
batch	Airflow: historical ingestion и суточные DAGs
analytics	Metabase
streaming	Flink JobManager, два TaskManager, checkpoint initializer, submitter
lakehouse	MinIO, REST catalog, Trino, Spark и отдельный upload client

Release публикует три project-owned образа: airflow, producer и neo4j. Neo4j образ содержит ClickHouse JDBC JAR. Flink собирается локально и сейчас не входит в GHCR release. MinIO и Spark также собираются локально и не входят в GHCR release. Остальные upstream infrastructure images используются напрямую; временные `*-ci` образы не являются release packages.

4. Установка с чистого состояния

Команды установки и dbt выполняются из корня репозитория в Bash.

```
git clone https://github.com/KonstantinLofichenko/ais-graph-analytics.git
cd ais-graph-analytics
cp .env.example .env
mkdir -p data/hais
cp neo4j/connectors/ais-sink.example.json neo4j/connectors/ais-sink.json
docker compose up -d --wait --wait-timeout 300
./scripts/bootstrap.sh
docker compose --profile streaming up -d
```

До запуска задайте локальные ClickHouse/Neo4j credentials, параметры connector и, если нужен live producer, BarentsWatch credentials. Внутри Kafka Connect Neo4j URI должен быть bolt://neo4j:7687; localhost относится к самому контейнеру. Не публикуйте .env, connector passwords или expanded Compose config.

Bootstrap проверяет connector, создаёт Kafka ais.positions, losslessly создаёт/ мигрирует derived analytics tables, применяет migrations 003-010, Neo4j constraints и регистрирует sink connector. Migration 001 предназначена для existing installation и не входит в fresh bootstrap. Analytics/dbt tasks bootstrap не запускает. Полная последовательность, включая live/batch/analytics profiles, в root quickstart.

5. Модель данных ClickHouse

5.1 Каноническая история

raw.ais_positions - ReplacingMergeTree(ingested_at), sorting key (mmsi, msgtime), месячные UTC event-time partitions. Logical validation replacement convergence использует FINAL. Live и historical могут перекрываться; перед backfill проверяйте coverage. raw.hais_positions сохраняет HAIS source values, в том числе sentinels и duplicates, а dbt staging нормализует их.

5.2 Flink-derived streams

Kafka topic	Persistent analytical table	Consumer group
ais.vessel.features	analytics.ais_vessel_features	clickhouse_ais_vessel_features
ais.vessel.gaps	analytics.ais_vessel_gap_events	clickhouse_ais_vessel_gaps

Persistent destinations используют MergeTree; Kafka Engine tables и MV остаются в raw. Эти transport objects не являются BI источниками. Flink consumer groups и ClickHouse consumer groups независимы. SQL ingestion использует literal topics и ais-kafka:29092, а не динамическое чтение .env: переименование topics требует отдельного изменения ClickHouse DDL.

activity_date - самая первая колонка обеих persistent tables. Features:

`toDate(window_end)`. Gaps: AIS_GAP_DETECTED использует `toDate(gap_detected_at)`, AIS_GAP_ENDED использует `toDate(gap_ended_at)`. Dates в UTC. Это DEFAULT-выражения ClickHouse, а не новые поля Flink payload; существующая история получает значение при чтении без полного rewrite. Vessel name хранится в features; старые записи без имени сохраняют NULL.

Migration 04_flink_derived.sh переносит legacy persistent tables из raw в analytics через RENAME с сохранением Atomic UUID/history. Конфликт одновременно существующих legacy и target tables вызывает ошибку до изменений. MV retarget выполняется контролируемо, Kafka group offsets сохраняются; rerun не копирует историю повторно. Точное описание schemas, float parsing, timestamp guards и monitoring: ClickHouse README.

5.3 Port visits и graph history

analytics.port_visits и analytics.port_graph_metrics используют replacement/ tombstone versions. Для active logical state нужен FINAL и is_deleted=0. Graph snapshot_date - UTC date window_start, не дата export; activity_date в dbt соответствует бизнес-дню. Физические retry versions не равны отдельным событиям.

6. dbt

```
python3.11 -m venv .venv-dbt
source .venv-dbt/bin/activate
python -m pip install -r dbt/requirements.txt
cp dbt/profiles.yml.example dbt/profiles.yml
export DBT_PROFILES_DIR="$PWD/dbt"
export CLICKHOUSE_USER=default
# Введите пароль локально, не сохраняйте его в документации.
read -r -s -p 'ClickHouse password: ' DBT_ENV_SECRET_CLICKHOUSE_PASSWORD
printf '\n'
export DBT_ENV_SECRET_CLICKHOUSE_PASSWORD
dbt deps --project-dir dbt --profiles-dir dbt
dbt parse --project-dir dbt --profiles-dir dbt
dbt debug --project-dir dbt --profiles-dir dbt
```

Закреплены dbt Core 1.12.5 и dbt-clickhouse 1.10.3.

Model	Назначение
vessels	Идентичность и текущее аналитическое состояние

Model	Назначение
vessel_daily_features	Суточные movement/quality features
vessel_daily_anomalies	Детерминированные anomaly candidates
vessel_daily_enriched	Features и только актуальные AI responses
current_port_visits	Logical active visits
port_graph_metrics_enriched	Метрики и reference dimensions
port_graph_communities	Аналитика communities

AIS code mappings - seeds. Country-dependent модели требуют raw.countries от optional Airbyte, обычный HAIS/canonical workflow от него не зависит. Детали incremental/history и точные команды: dbt README.

7. Сохранённый legacy historical ClickHouse путь

Файлы hais_YYYY-MM-DD.snappy.parquet помещаются вручную в data/hais либо HAIS_HOST_DIR. Один каталог монтируется read-only в Airflow и ClickHouse. Файлы неизменны во время load; неполные downloads нельзя публиковать под финальным именем. Manual DAG ais_hais_historical_ingestion проверяет весь диапазон до вставок; успех определяется file_name + actual file_size. Successful versions пропускаются. Audit сохраняет running/success/failed; prior attempt без success требует operator review, автоматические task retries отключены. Raw insert и audit не атомарны; partial insert нельзя слепо перезагружать.

Операция	Начало	Конец
HAIS ingestion	Inclusive calendar date	Inclusive calendar date
Canonical dbt load	Inclusive	Exclusive UTC midnight
Daily analytics	UTC activity date	Следующая UTC граница

Например, ingestion September 1-16 заканчивается 2026-09-16, canonical macro использует end_date 2026-09-17. Historical source может быть беднее live по name/type, поэтому overlapping backfill требует проверки attributes. HAIS workflow и отдельный Historic REST repair не входят в daily master.

8. Ежедневная оркестрация Airflow

Airflow 3.3.1 standalone - local runtime, отдельный persistent metadata volume. Для production нужны отдельные сервисы и подходящая metadata database. Основной DAG daily_ais_pipeline запускается в 02:00 UTC, catchup=False, max_active_runs=1. Фактический порядок:

```
resolve_activity_date -> dbt_core -> dbt_vessel_daily_features
  -> ais_port_visits -> ais_gds_metrics -> ais_graph_metrics_export
  -> dbt_graph_models -> dbt_vessel_daily_anomalies -> ai_enrichment
  -> dbt_vessel_daily_enriched -> dbt_tests
```

activity_date берётся явно или как вчера UTC от actual master start и фиксируется один раз. Graph children manual, но master запускает их по очереди и ждёт success; они должны быть unpaused. Failure блокирует downstream stages. Повторный запуск для historical date требует той же явной даты; graph writers не должны перекрываться.

```
for dag in ais_port_visits ais_gds_metrics ais_graph_metrics_export daily_ais_pipeline; do
  docker compose exec -T airflow airflow dags unpause "$dag"
done
docker exec ais-airflow airflow dags trigger daily_ais_pipeline \
  --conf '{"activity_date":"2026-09-27"}'
```

Image использует COPY: изменения DAG/pipeline требуют rebuild,

`docker compose --profile batch up -d --build airflow`. Полное руководство Airflow.

9. AI enrichment

AI объясняет selected anomalous vessel-days; deterministic features и anomaly selection остаются вне модели. Eligible ranks только 1-100. History table analytics.vessel_ai_enrichment хранит model, prompt version, exact input hash, response ID, token counts и creation time. Cache lookup требует точного совпадения date/model/prompt/input hash; AI_ENRICHMENT_LIMIT ограничивает новые вызовы после cache hits. Rank 101+ не становится eligible при cache hits. Prompt version: vessel_anomaly_v1. FINAL mart показывает только current input matches.

CI не вызывает OpenAI. Для refresh presentation без API calls используйте AI freshness workflow. History responses сохраняются даже при изменении input.

10. Neo4j и графовая аналитика

```
(Vessel)-[:VISITED]->(Port)
(Port)-[:CONNECTED_TO]->(Port)
(Port)-[:MEMBER_OF]->(Community)
```

Kafka Connect обновляет текущие Vessel nodes по MMSI. Visits workflow публикует сводки Vessel-to-Port и transitions между последовательными detected visits. CONNECTED_TO не доказывает отсутствие ненаблюдаемых посещений между точками. GDS: directed PageRank с movementCount weight, damping 0.85, до 20 iterations; undirected weighted Louvain. Метрики записываются в active Ports и экспортируются в ClickHouse. Isolated ports не входят в active projected graph.

Snapshot lineage связывает visitRunId и connection run. Mismatch, unknown ports или incomplete metrics блокируют export. Valid empty snapshot поддерживается. Community names выбираются по представительным портам, technical community IDs не стабильны между независимыми Louvain runs. Neo4j хранит current state, ClickHouse - time-versioned history; JDBC bridge используется для diagnostics, а не обычного ingestion. GDS operations и graph export.

11. Metabase

Dashboard Norway Port Graph Analytics содержит шесть tabs и 31 saved card. Versioned export хранит dashboard layout, mappings и отдельные card JSON.

Tab	Содержание
Ports on Map	Port map и recovered-gap vessel ranking
Ports	Centrality и communities
Vessels	Activity, categories, last-known positions
Anomalies & AI Insights	Daily anomaly candidates и AI responses
Near Real-Time Vessel Analytics	Последние completed Flink windows, speed/category/status
Near Real-Time AIS Gap Monitoring	Detected/recovered gaps, durations, rankings и lifecycle events

Feature queries сначала выбирают window_minutes, затем общий max(window_end) этой ветки, а не latest row отдельно для каждого MMSI. Window minute selector 5/15/30/60 mapped на cards 58-62; details table card 63 сохраняет собственный query default 5. Average speed card - unweighted average per-vessel averages.

Gap cards: detected/recovered за последний час, recovered vessel ranking и average/distribution duration за 24 часа, latest lifecycle events за 24 часа. Card 70 с названием Latest Gap Events в Ports on Map фактически показывает ranking recovered gaps; настоящий latest events table - card 71. Документация описывает export как он есть; live dashboard этим обновлением не меняется.

Новые tabs используют latest windows и rolling now UTC, независимо от historical Date и Last Seen. activity_date полезна для history, но не основной live filter. Metabase читает persistent ClickHouse tables, а не Kafka Engine transport. Детали cards и export/import commands: Metabase README.

12. CI

Шесть jobs проверяют конфигурацию и implementation в isolated окружении.

Job	Проверка
Shell and Compose	Syntax и все profiles
Pipeline tests	Airflow build и isolated DAG/pipeline tests
Producer	Python tests и image build
dbt + ClickHouse	Fresh init/migrations, fixtures, seeds, models/tests
ClickHouse + Kafka	Derived ingestion, analytics ownership, names, idempotent migration
Kafka + Neo4j	Topic produce/consume, custom Neo4j constraints/write/read

raw.countries fixture принадлежит CI и не должна попасть в production init. CI не зависит от live BarentsWatch, OpenAI, Airbyte control plane, локального Mac или production Metabase. Flink recovery integration запускается отдельно: Flink tests.

13. Release workflow

Stable semantic tag vMAJOR.MINOR.PATCH запускает release: проверка формата и принадлежности commit к main, reusable CI, build трёх project-owned images, push в GHCR и GitHub Release с generated notes. Image tags: MAJOR.MINOR.PATCH, MAJOR.MINOR, MAJOR, latest. Опубликованный tag не перемещают, выпускают новый. Это описание workflow, а не инструкция выполнить release в текущей задаче.

14. Опубликованные containers

```
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-airflow:latest
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-producer:latest
docker pull ghcr.io/konstantinlofichenko/ais-graph-analytics-neo4j:latest
```

GHCR публикует только эти project-owned images; Flink пока local build. Upstream images не перепубликуются как project packages.

15. Проверка и эксплуатация

```
./scripts/status.sh
docker compose --profile streaming config --quiet
docker compose ps
docker compose logs --tail 100 airflow
docker compose logs --tail 100 flink-job-submitter
docker compose exec -T ais-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server ais-kafka:29092 --list
curl -fsS http://localhost:8082/jobs/overview
```

Проверьте две RUNNING jobs, completed checkpoints, оба output topics и прирост persistent ClickHouse tables. Для логической проверки ReplacingMergeTree нужен FINAL, derived MergeTree не дедуплицирует replay автоматически. Repeat submitter не должен создавать дополнительные jobs. State recovery tests проверяют pending и active gap timers, partial windows и concurrent submission, а не только status.

16. Важные правила качества данных

- Coverage ship type/name зависит от источника; NULL нельзя заменять вымышленной идентичностью.
- Navigation status - noisy evidence, а не окончательная классификация движения.
- Speed metrics исключают implausible values по установленным validity rules.
- Stationary percentage основан на observations, а не elapsed-time interpolation.
- AI описывает engineered inputs, а не поведение между наблюдениями.
- Community ID технический; human-readable label описывает текущий snapshot.
- EXACTLY_ONCE checkpoint state не устраняет output duplicates при replay.

17. Известное ограничение: фрагментация port visit на границе полуночи

Batch port-visit detector работает в bounded UTC windows и не переносит своё состояние между суточными запусками. Одна физическая стоянка через полночь может дать два daily fragments: overcount visits, fragmented duration и отличия graph metrics. Flink checkpoints для live AIS jobs не исправляют эту отдельную batch семантику. Возможные решения - persistent detector state либо continuous physical visits с последующим daily projection; это отдельная работа.

18. Troubleshooting

При container name conflict сначала `docker ps -a`, затем удаляйте только obsolete manual/test containers. Kafka Connect обращается к bolt://neo4j:7687, не localhost. Healthchecks Confluent services используют TCP там, где curl недоступен. Custom Neo4j image содержит ClickHouse JDBC и проверяется в CI.

dbt connection: проверьте active virtualenv, DBT_PROFILES_DIR, host/port и DBT_ENV_SECRET_CLICKHOUSE_PASSWORD. Flink: проверьте submitter logs, зарегистрированные TaskManagers, free slots, Kafka handshake, completed snapshots и shared mount. Restore failure требует устранить причину; нельзя молча сбрасывать state.

19. Безопасная остановка и cleanup

```
docker compose --profile '*' down
```

Не добавляйте -v, если данные должны сохраниться. Named registry volume и bind checkpoint directory нужны для recovery. Для restart только Flink используйте команды раздела 2.3, а не shutdown всей платформы. Не удаляйте raw history или используемые checkpoints. Старые `*-ci` images можно удалить после проверки, что это disposable test artifacts.

20. Карта документации репозитория

Path	Назначение
README.md	Обзор и fresh-clone quickstart
docs/README.md, docs/README.ru.md	Английский/русский operational guide
docs/assets/README.md	Общая диаграмма и regeneration
flink/README.md	Jobs, automatic startup, checkpoints и recovery
clickhouse/README.md	Derived ingestion, schemas, monitoring
metabase/README.md	Шесть tabs, cards и export/import
airflow/README.md	Orchestration и batch operations
dbt/README.md	Models, seeds и canonical load
pipelines/hais/README.md	HAIS file/audit workflow
pipelines/historic_ais/README.md	Historic REST repair
pipelines/port_visits/README.md	Visit detection и publication
pipelines/graph_metrics/README.md	Graph export/history
neo4j/README.md, neo4j/gds/README.md	Graph и GDS
airbyte/README.md	Optional country reference export
data/ports/README.md	Датированный pilot reference

21. Текущее состояние и roadmap

Реализованы live/historical ingestion, canonical ClickHouse history, dbt models, Airflow daily orchestration, Neo4j/GDS, deterministic anomalies, cached AI enrichment, durable Flink windows/gap timers, шесть Metabase tabs, шесть CI jobs, semantic releases и GHCR для airflow/producer/neo4j. MinIO, Iceberg REST, PySpark, Trino и manual historical DAG реализованы локально. dbt-trino, incremental lakehouse и production distributed catalog/storage остаются roadmap.

Перегенерация PDF

Английский PDF строится из docs/README.md, русский - из этого файла. Оба используют одну architecture.png. При изменении платформы обновляйте оба Markdown sources, при изменении диаграммы сначала запускайте её renderer.

```
python3 -m venv /tmp/ais-docs-venv
/tmp/ais-docs-venv/bin/pip install reportlab pillow pymupdf
/tmp/ais-docs-venv/bin/python docs/render_pdfs.py
```

Renderer использует Arial на macOS или DejaVu на Linux. После regeneration визуально проверьте все страницы, Cyrillic glyphs, tables, code wrapping, architecture labels и recovery/delivery limitations. Датированные screenshots и validation results сохраняют собственные provenance dates.